

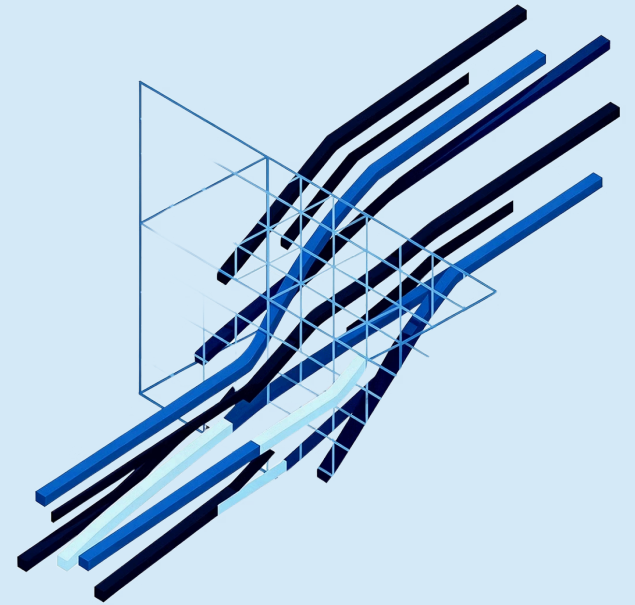
Reducción de Dimensionalidad con PCA

MÓDULO DE MACHINE LEARNING · DIPLOMADO DATA ENGINEER

Eduardo Castillo Rodríguez

Máster en Inteligencia Artificial

Dataset de ventas retail — **23 millones de registros**



Antes de entrenar un modelo... ¿Qué queremos describir?

Pregunta de negocio	Observación	Características
¿Cómo se comporta una tienda?	Tienda + Semana	Productos
¿Cómo se comporta un producto?	Producto + Semana	Tiendas
¿Cómo se comporta un cliente?	Cliente	Variables del cliente

Toda matriz de características comienza definiendo qué queremos describir.

El Dataset Original

PUNTO DE PARTIDA

Semana	Tienda	Producto	Ventas	Promoción
1	1	1001	15	Sí
1	1	1002	8	No
1	1	1003	21	Sí
1	2	1001	10	Sí
1	2	1002	5	No

23 millones de registros con esta estructura

¿Qué representa una fila?

Transacción de un producto en una tienda durante una semana.

¿Puede un modelo de Machine Learning aprender el comportamiento de una tienda con una sola fila?

Una fila representa únicamente la venta de un producto específico durante una semana en una tienda. Con esa información no podemos describir el comportamiento completo de la tienda. Para caracterizar una tienda necesitamos observar simultáneamente las ventas de todos sus productos.

Una transacción no describe una tienda.

¿Puede un modelo de Machine Learning trabajar directamente con 23 millones de registros?

EL PROBLEMA DE REPRESENTACIÓN

Los datos son transaccionales

Cada fila registra un evento puntual de venta, no un comportamiento.

Una fila $\neq$ un comportamiento

Una transacción no describe cómo se comporta una tienda a lo largo del tiempo.

Se necesita una representación adecuada

Los datos deben transformarse antes de entrenar cualquier modelo predictivo.

VEREDICTO

No.

Los modelos de Machine Learning no aprenden directamente de los datos transaccionales. Aprenden de la representación que construimos para describir el problema.

Ingeniería de Características: De Transacciones a Observaciones

El objetivo del Pivot no es simplemente reorganizar los datos. El objetivo es construir una nueva representación donde cada fila describa completamente el comportamiento de una tienda durante una semana.

1

Dataset transaccional

23M registros · Semana / Tienda / Producto / Ventas

2

Agrupación semanal

Suma de ventas por Semana + Tienda + Producto

3

Pivot

Cada producto se convierte en una columna

4

Matriz de características

2,322 observaciones × 5,000 variables

La unidad de análisis **deja de ser una transacción** y pasa a ser el **comportamiento semanal de una tienda**.

23M

Registros originales

Filas en la tabla transaccional cruda

2,322

Observaciones estructuradas

Combinaciones únicas Semana × Tienda

La Matriz de Características: 2,322 × 5,000

Ahora cada fila representa una tienda durante una semana.

Los productos ya no representan registros. Ahora representan características utilizadas para describir el comportamiento de una tienda.

Observación	Prod1	Prod2	Prod3	Prod4	...
Sem1-T1	120	0	0	15	...
Sem1-T2	0	85	0	0	...
Sem2-T1	40	0	18	0	...
Sem2-T2	0	0	0	62	...
...	...	...	...	...	...

Matriz altamente dispersa (**Sparse Matrix**): la mayoría de los valores son cero.

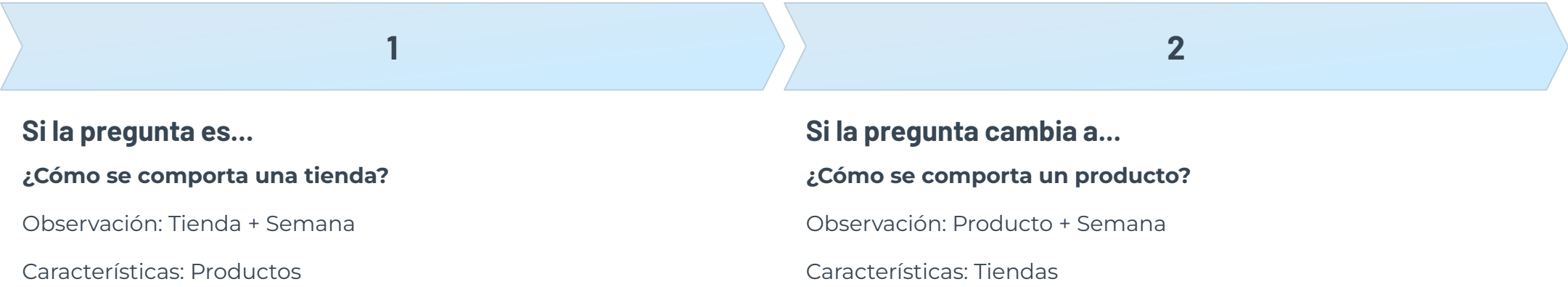


La Matriz de Características Depende de la Pregunta de Negocio

PRINCIPIO FUNDAMENTAL DE INGENIERÍA DE CARACTERÍSTICAS

Pregunta de negocio	Observación	Características
Describir el comportamiento de una tienda	Tienda + Semana	Productos
Describir el comportamiento de un producto	Producto + Semana	Tiendas
Describir el comportamiento de un cliente	Cliente	Variables del cliente
Describir el comportamiento de un paciente	Paciente	Variables clínicas

Un mismo dataset puede generar distintas matrices de características



La matriz de características no está determinada por el dataset. Está determinada por la pregunta de negocio que queremos responder.

No existe una única matriz de características.

La matriz depende completamente de la pregunta que queremos responder.

Nuestro caso de estudio:

Pregunta → ¿Cómo se comporta cada tienda semana a semana?

Observación → Tienda + Semana

Características → 5,000 productos

¿Es buena idea entrenar un modelo con 5,000 variables?

PREGUNTA DE NEGOCIO

¿Necesitamos realmente las ventas de los 5,000 productos para describir el comportamiento de una tienda?

EL PROBLEMA DE DIMENSIONALIDAD

Maldición de la dimensionalidad

Con 5,000 variables, los datos se dispersan en el espacio de características y los algoritmos pierden capacidad de generalización.

Ruido vs. señal

Variables correlacionadas y redundantes hacen que el modelo aprenda ruido en lugar de patrones reales.

Costo computacional

Miles de variables ralentizan el entrenamiento y amplifican el sobreajuste.

Interpretabilidad

Imposible identificar qué variables importan entre 5,000 candidatas simultáneas.

Sparsity

La mayoría de valores son cero — productos sin venta en ciertas tiendas amplifican el ruido.

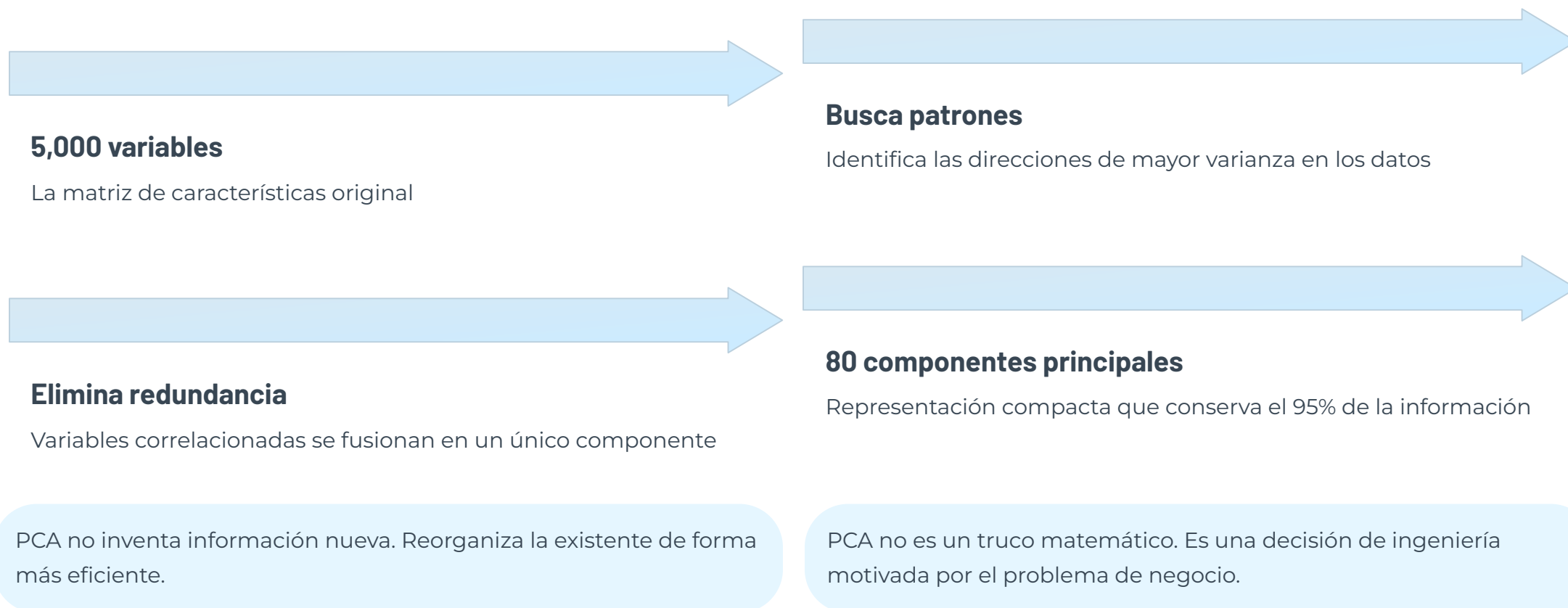
❏ **La respuesta es no.** Necesitamos reducir la dimensionalidad sin perder información relevante.

¿Qué hace realmente PCA?

REDUCCIÓN DE DIMENSIONALIDAD — SIN MATEMÁTICAS

PCA no reduce únicamente el número de variables. Construye una representación más compacta de las observaciones.

En nuestro caso, cada observación corresponde al comportamiento semanal de una tienda.



¿Qué representa realmente una Componente Principal?

CONCEPTO CLAVE

**Una componente principal NO representa un producto.
Representa un patrón de comportamiento.**

$PC1 = 0.18 \times \text{Producto A}$
 $+ 0.07 \times \text{Producto B}$
 $- 0.11 \times \text{Producto C}$
 $+ 0.04 \times \text{Producto D}$
 $+ \dots (5,000 \text{ productos})$

No es un producto

Ningún componente equivale a un SKU individual.

Es una combinación

Mezcla miles de productos con pesos positivos y negativos.

Es un patrón

PC1 podría representar 'tiendas que venden más en temporada alta'.

- ❏ Una componente principal = un patrón de comportamiento, no un producto.

¿Por qué debemos escalar los datos?

EL PROBLEMA DEL ESCALAMIENTO

Producto A

Ventas: 0 – 10 unidades

Producto B

Ventas: 0 – 10,000 unidades

Sin escalamiento → Producto B domina el análisis, ignorando el Producto A

StandardScaler

Transforma cada variable a media = 0 y desviación estándar = 1.

Todas las variables tienen igual importancia antes de PCA.

Sin escalamiento, PCA mide magnitud — no varianza.

PASO OBLIGATORIO ANTES DE PCA

PCA No Aparece por Casualidad – Aparece para Resolver un Problema Real

El Análisis de Componentes Principales (PCA) es la solución natural al problema de dimensionalidad — no un truco matemático, sino una **decisión de ingeniería motivada por el negocio**.



Compresión inteligente

Comprime 5,000 variables en pocos componentes, preservando la información crítica del dataset



Elimina redundancia

Variables correlacionadas se fusionan en un único componente, sin pérdida crítica de información



Máxima varianza

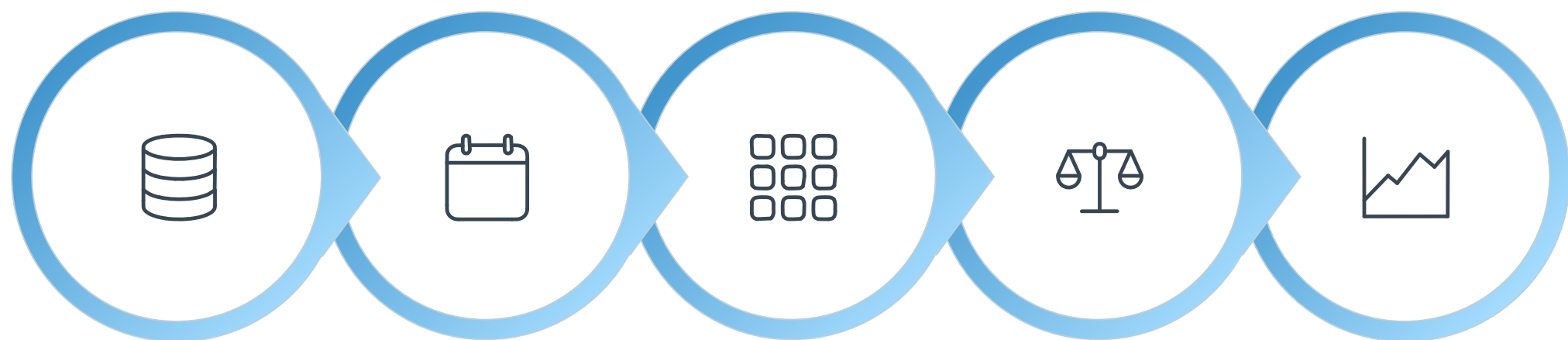
Identifica las **direcciones de mayor varianza** y proyecta las observaciones sobre ellas



Prepara el terreno

Los componentes resultantes son **entrada ideal** para cualquier algoritmo de Machine Learning supervisado o no supervisado

El Flujo Completo: De 23 Millones de Registros a un Modelo de Machine Learning



23M registros

Agregación
semanal

Matriz pivot

Escalamiento

PCA 80

Escalamiento

Paso **obligatorio** antes de PCA — ninguna variable debe dominar artificialmente por escala

80 componentes

Determinado por **varianza explicada acumulada**, no por regla fija

Factor 62.5×

De 5,000 a 80 variables — compresión masiva preservando el **95% de la información**

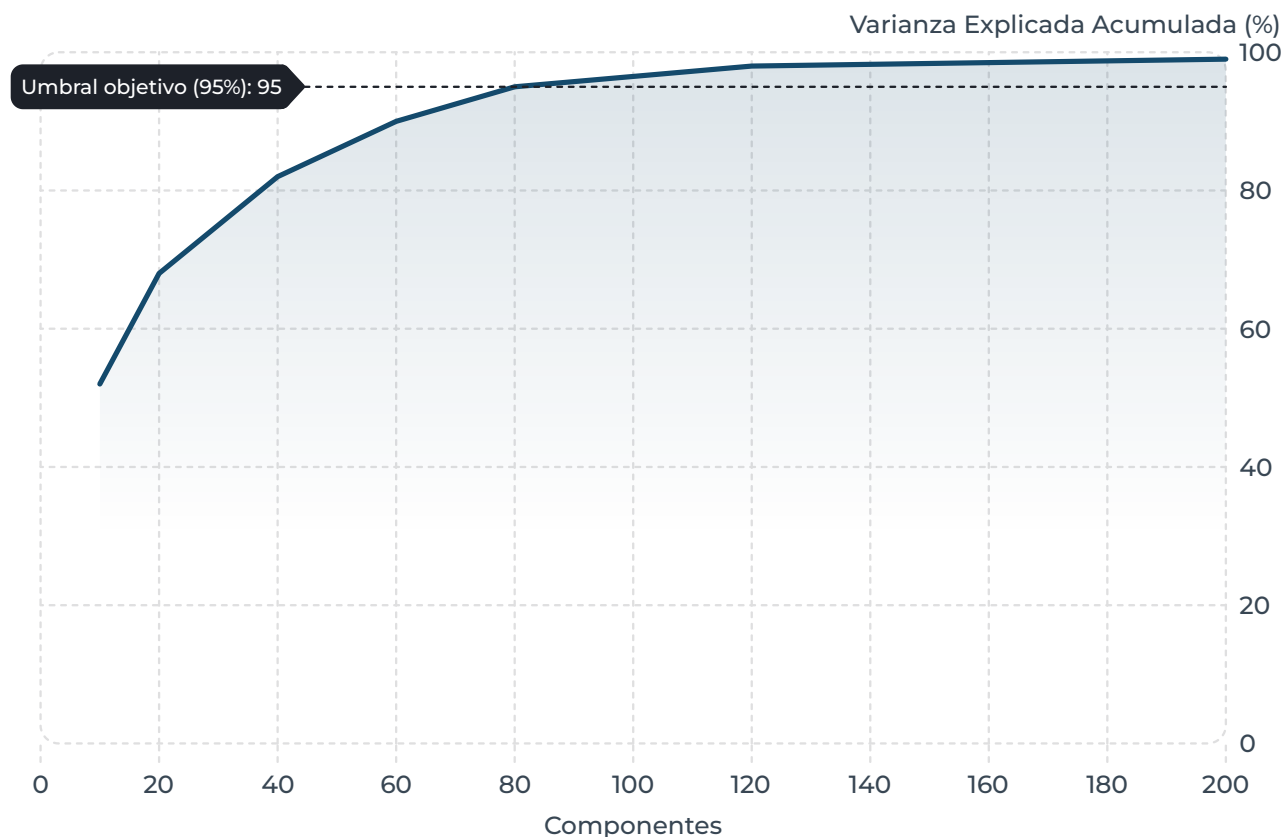
Resultados de PCA: 5,000 Variables → 80 Componentes

La reducción de dimensionalidad produce resultados concretos y medibles sobre el dataset de ventas retail.

"Los datos determinan el número de componentes."

No elegimos arbitrariamente 80 componentes — el número se obtiene analizando la curva de varianza explicada acumulada.

⚠ GRÁFICO CONCEPTUAL — REFERENCIAL



Curva de varianza explicada acumulada (conceptual)

i La curva real será obtenida durante la implementación en Google Colab.

Componentes 1 y 2

Solo para **visualización** 2D — no para entrenar el modelo

Componentes 3-80

Alimentan el modelo predictivo con representación compacta y densa

Interpretación

Cada componente es una **combinación lineal** de los 5,000 productos y captura un patrón de comportamiento de las observaciones.

Beneficio práctico

Entrenamiento más rápido y **menor riesgo de sobreajuste**

Visualización: Los Primeros 2 Componentes Revelan Estructura Oculta

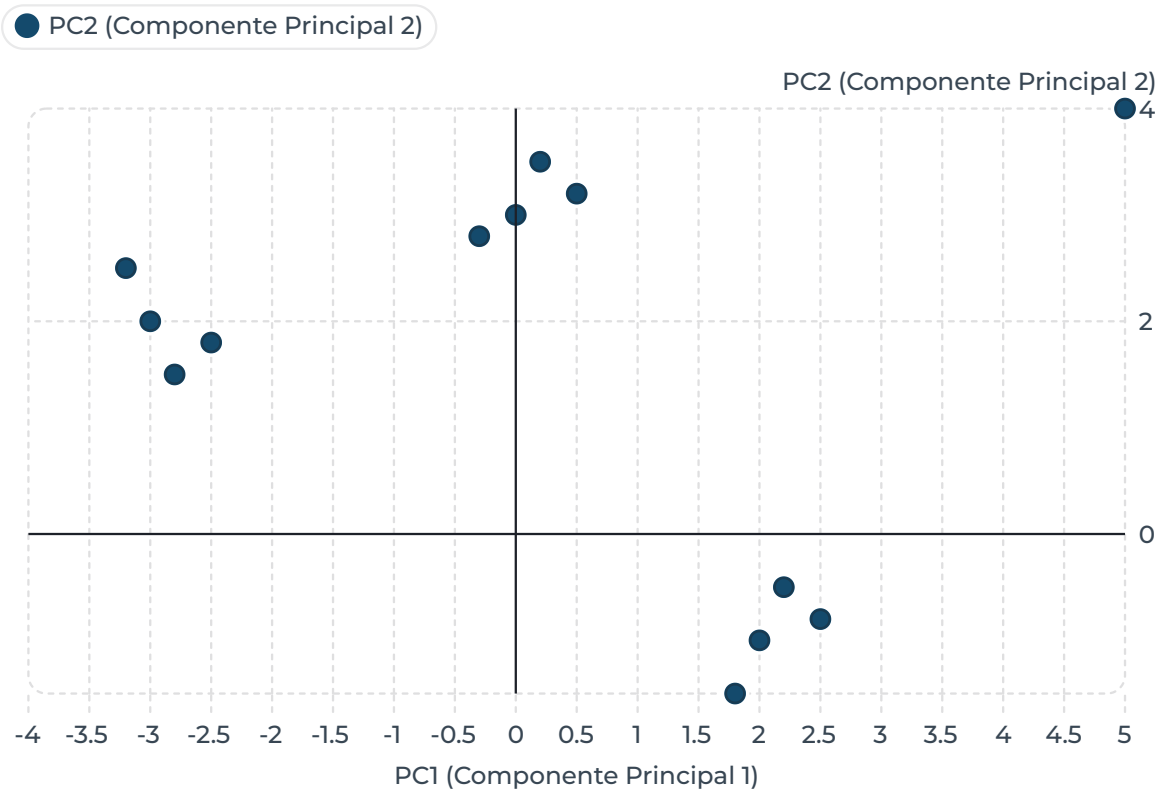


Gráfico conceptual — 2,322 observaciones proyectadas en 2D

Patrones de comportamiento similares

Tiendas con patrones de comportamiento similares aparecen próximas en el espacio reducido.

Detección de anomalías

Observaciones atípicas se identifican **antes del modelado**.

Validación del pipeline

Si hay estructura en los datos, PCA la hace visible. Si no, el modelo predictivo tampoco la encontrará.

❗ Esta visualización es el **primer entregable** del notebook en Google Colab.

La Lección Central: El Trabajo Real de Machine Learning No Es Entrenar Modelos

El mito

El valor está en **elegir el algoritmo correcto** — Random Forest vs. XGBoost vs. redes neuronales

La realidad

El **80% del trabajo** está en construir una representación adecuada de los datos — antes del primer entrenamiento

23M

Registros de origen

Dataset transaccional crudo de ventas retail

80

Componentes finales

Listos para modelos predictivos, preservando el 95% de la información

Lo que hicimos hoy

1 Transformamos 23M de registros

En 2,322 observaciones estructuradas mediante agregación y pivoteo

2 Redujimos 5,000 variables a 80

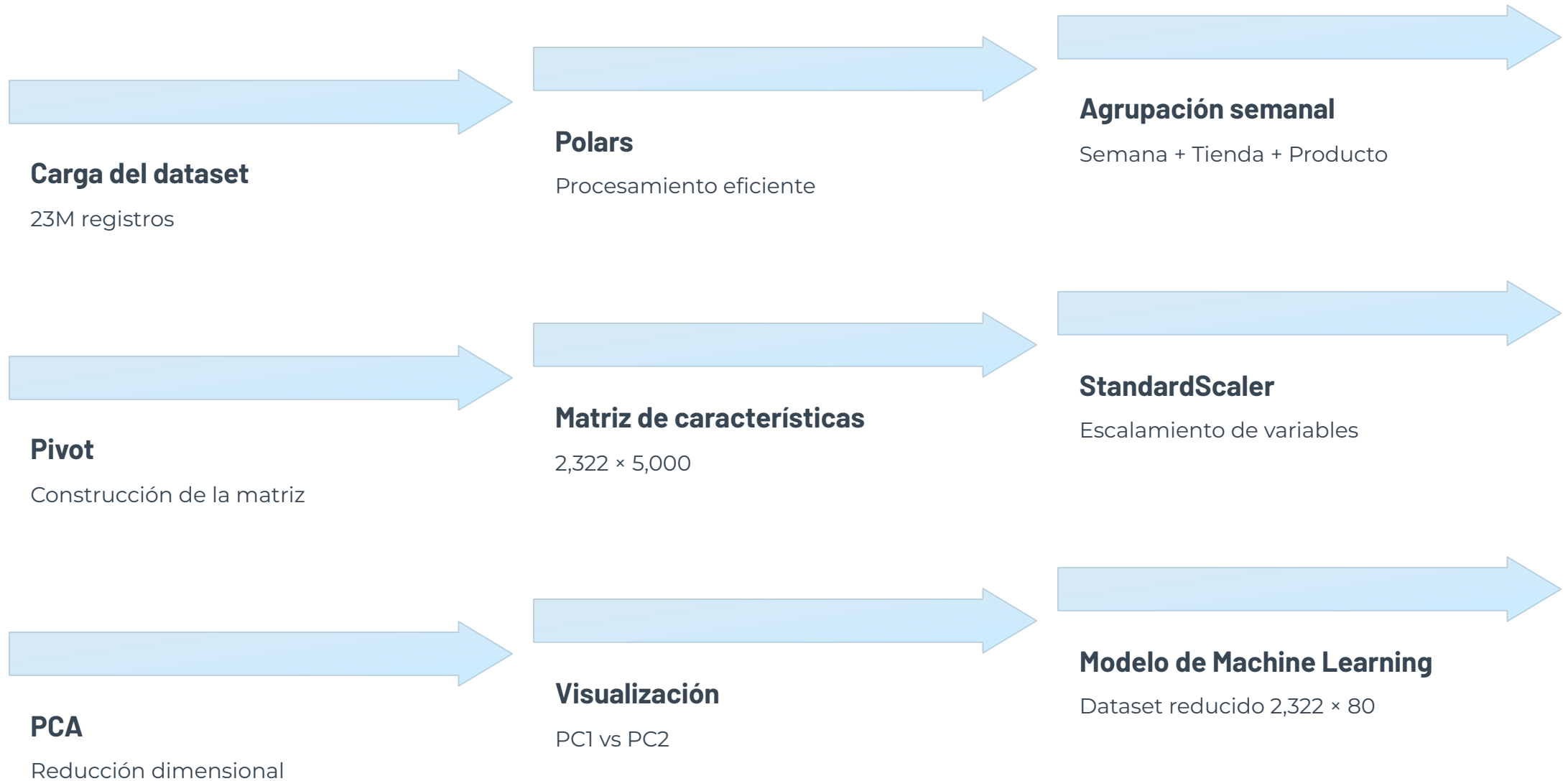
Compresión 62.5× conservando el 95% de la información

3 Patrones de comportamiento visibles

Agrupamientos naturales de tiendas visibles en el espacio reducido — validación visual del pipeline

Próximo paso: en Google Colab implementaremos este flujo completo — desde la carga del dataset hasta la aplicación de PCA.

Lo que implementaremos en Google Colab



 Al finalizar la sesión, cada estudiante tendrá un dataset de **2,322 × 80** listo para entrenar modelos predictivos.

El Machine Learning comienza mucho antes del primer algoritmo.

Un modelo predictivo mediocre sobre datos bien representados **supera consistentemente** a un modelo sofisticado sobre datos mal estructurados.

El valor real está en construir una representación adecuada. Los algoritmos aprenden sobre representaciones, no sobre datos transaccionales.

Representación primero

Antes de elegir un algoritmo, verifica que tus datos tengan la forma correcta para que el modelo aprenda algo útil.

PCA como diagnóstico

La visualización en 2D valida que la ingeniería de características produjo estructura real, no solo estética.

Compresión sin pérdida crítica

62.5× de reducción manteniendo el 95% de varianza — el balance que define proyectos reales.



Todo proyecto de Machine Learning comienza respondiendo una pregunta muy simple: ¿Qué queremos describir?

i Cuando definimos correctamente la observación y construimos una buena representación de los datos, los algoritmos pueden aprender.

¿Cómo se implementa este pipeline en Python?

ENTRENAMIENTO

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier
import joblib

# 1. Escalar los datos
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# 2. Aplicar PCA
pca = PCA(n_components=50)
X_train_pca = pca.fit_transform(X_train_scaled)

# 3. Entrenar el modelo
model = RandomForestClassifier()
model.fit(X_train_pca, y_train)

# 4. Guardar los 3 objetos
joblib.dump(scaler, 'scaler.pkl')
joblib.dump(pca, 'pca.pkl')
joblib.dump(model, 'model.pkl')
```

PREDICCIÓN EN PRODUCCIÓN

```
import joblib

# 1. Cargar los 3 objetos guardados
scaler = joblib.load('scaler.pkl')
pca = joblib.load('pca.pkl')
model = joblib.load('model.pkl')

# 2. Nuevo registro con 5,000 variables
X_nuevo = obtener_nuevo_registro()

# 3. Aplicar el MISMO scaler del entrenamiento
X_scaled = scaler.transform(X_nuevo)

# 4. Aplicar el MISMO PCA del entrenamiento
X_pca = pca.transform(X_scaled)

# 5. Predecir
prediccion = model.predict(X_pca)
```

  Nunca uses `fit_transform()` en producción. Usa siempre `transform()` — el scaler y el PCA ya fueron ajustados durante el entrenamiento. Aplicar `fit_transform()` sobre datos nuevos recalcularía los parámetros y corrompería la predicción.

Regla de oro

`fit_transform()` → solo en entrenamiento · `transform()` → siempre en producción

PCA en un Pipeline de Machine Learning

MÓDULO DE MACHINE LEARNING · DIPLOMADO DE BIG DATA

FLUJO DE ENTRENAMIENTO



FLUJO DE PREDICCIÓN



El modelo nunca recibe las variables originales directamente. Antes de realizar una predicción, los nuevos datos deben pasar por el mismo StandardScaler y el mismo PCA utilizados durante el entrenamiento.

Objetos que se almacenan en producción

Modelo PCA

StandardScaler

Modelo de Machine Learning